

CONTAINER ORCHESTRATION



Work Integrated Learning Programmes

MOHANA MURALI GURUNATHAN

AGENDA

VMs Vs Containers
Need for Container Orchestration
Kubernetes (K8s) Overview
Cloud Native Design Principles



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

WHY DO WE NEED CONTAINER ORCHESTRATION?



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

Container orchestration is necessary because running containers at scale in production is too complex to manage manually.

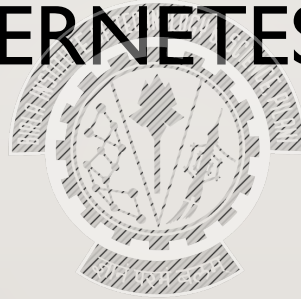
Platforms like Docker makes it easy to spin up a few containers on a single laptop, a real-world application might require **hundreds or thousands of containers** spread across dozens of virtual or physical servers.

Without orchestration, managing that environment would require a massive army of engineers to constantly monitor, restart, and scale systems 24/7. Container orchestration tools—like **Kubernetes**—automate these operational headaches.

Work Integrated Learning Programmes

Work Integrated Learning Programmes

KUBERNETES



Work Integrated Learning Programmes

Work Integrated Learning Programmes

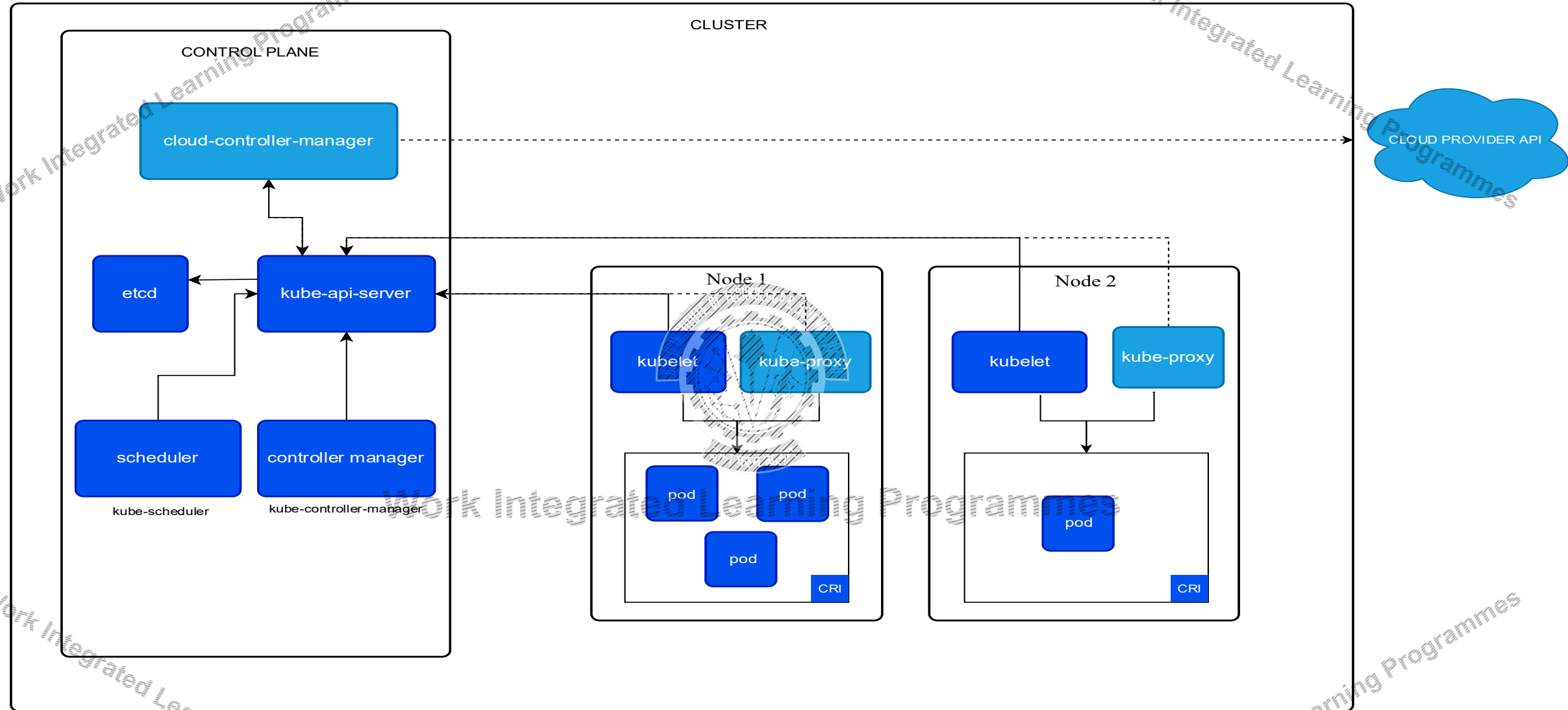
Work Integrated Learning Programmes

Kubernetes (K8s) - An open-source system for orchestrating containers

- Managing containerized applications
- Automating deployment: Dynamically pushing configuration files to running jobs
- Service discovery and load balancing
- Auto-scaling



kubernetes



[HTTPS://KUBERNETES.IO/DOCS/CONCEPTS/ARCHITECTURE/](https://kubernetes.io/docs/concepts/architecture/)

A Kubernetes cluster is divided into two primary layers: the control plane and the worker nodes.

The Control Plane: Consider this like a **Brain**. This manages the global state and scheduling decisions of the entire cluster.

- **kube-apiserver:** The front-end REST interface that validates and processes all internal and external cluster requests.
- **etcd:** A distributed, fault-tolerant key-value store acting as the ultimate source of truth for all cluster configuration and state data.
- **kube-scheduler:** Evaluates resource requirements and constraints to assign incoming pods to the most appropriate compute node.
- **kube-controller-manager:** Continuously monitors the cluster to ensure the actual running state matches the desired declared state.

Worker Nodes : Consider this like the **Muscle**. The physical or virtual machines that physically execute the application workloads.

- **Pods:** The smallest deployable object in Kubernetes, representing a single instance of an application running one or more tightly coupled containers.
- **kubelet:** An agent on each node communicating with the control plane to ensure containers are running correctly inside their assigned pods.
- **kube-proxy:** A network proxy that facilitates communication and traffic routing inside and outside the cluster.
- **Container Runtime:** The underlying engine (e.g., containerd, CRI-O) responsible for pulling images and executing the containers.

Work Integrated Learning Programmes

Work Integrated Learning Programmes

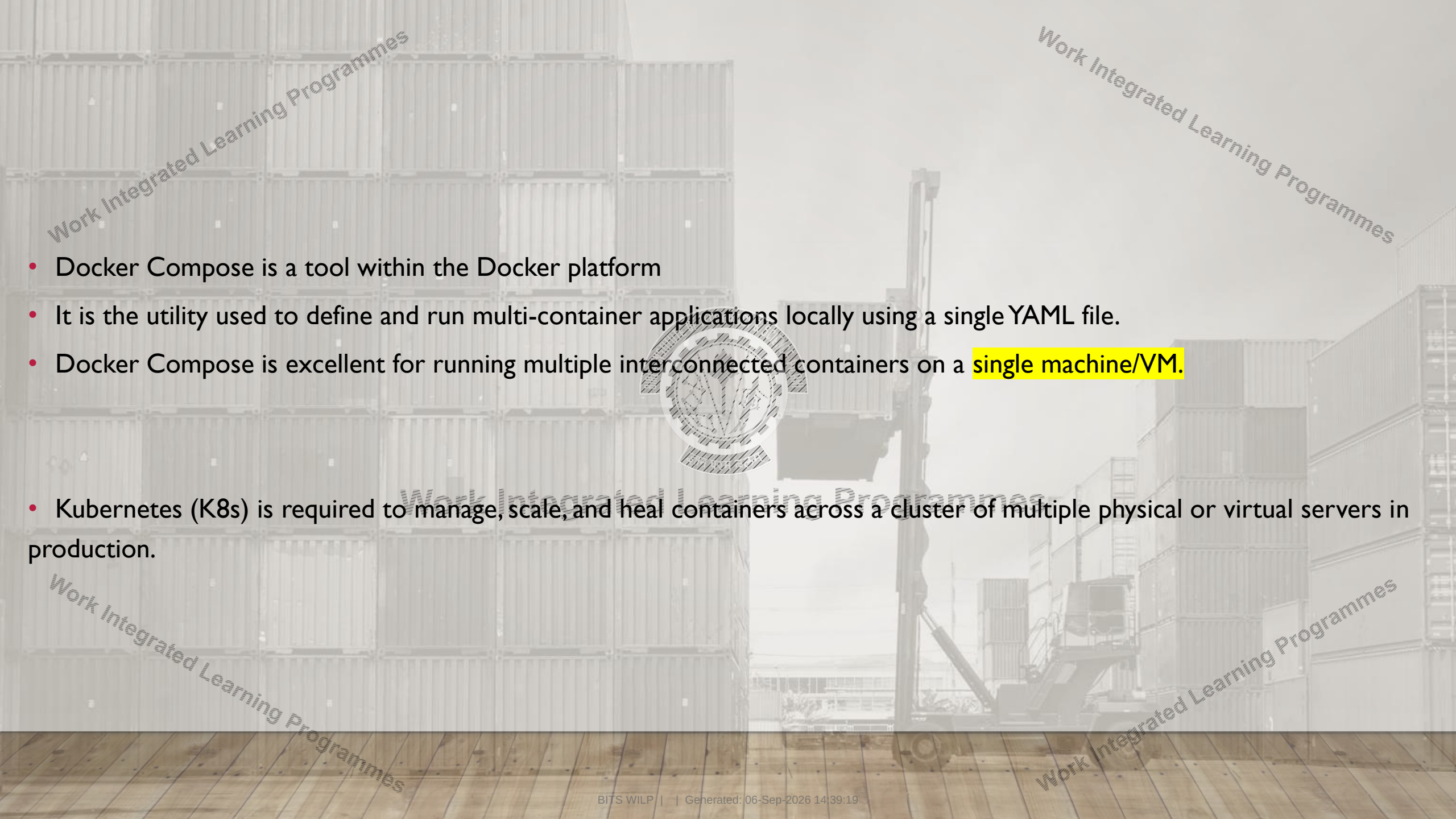
DOCKER COMPOSE & KUBERNETES



Work Integrated Learning Programmes

Work Integrated Learning Programmes

Work Integrated Learning Programmes

- 
- Docker Compose is a tool within the Docker platform
 - It is the utility used to define and run multi-container applications locally using a single YAML file.
 - Docker Compose is excellent for running multiple interconnected containers on a **single machine/VM.**
 - Kubernetes (K8s) is required to manage, scale, and heal containers across a cluster of multiple physical or virtual servers in production.

KUBERNETES VS. DOCKER COMPOSE

Multi-Node Clustering:

Docker Compose can only deploy containers to the single machine it is running on. If that server crashes, the entire application goes down.

Kubernetes: Groups dozens or hundreds of servers into a single cluster. K8s decides which server has the available memory and CPU to run a container, spreading the load automatically.

High Availability and Self-Healing:

If a container crashes or hangs due to a memory leak, Docker Compose won't automatically provision a new copy on a different, healthy server.

Kubernetes: Continuously monitors the infrastructure. If a container or an entire physical server dies, K8s instantly recreates those exact containers on a healthy server (self-healing).

Docker Compose lacks the infrastructure automation capabilities required to run critical applications in production

KUBERNETES VS. DOCKER COMPOSE

Dynamic Scaling and Load Balancing:

Docker Compose can scale up, but it requires manually changing the configuration or running a command to duplicate a container on the same machine. It does not natively manage external network traffic routing across multiple instances out of the box.

Kubernetes: Automatically scales the number of running containers up or down based on actual CPU and memory usage (autoscaling). It includes built-in load balancers to distribute incoming web traffic evenly across those containers.

Zero-Downtime Rolling Updates:

For the Docker Compose to update an application, it has to stop the old container and start the new one, causing a brief moment of downtime.

Kubernetes: Replaces old containers with new ones progressively (rolling updates). If the new version fails a health check, K8s automatically rolls back to the previous stable version without the users ever noticing.

Docker Compose lacks the infrastructure automation capabilities required to run critical applications in production